SPI on FPGA: From the Bus to Verified RTL

Verilog | SystemVerilog | VHDL | Testbenches | Reproducible CLI Testing

Sipeed FPGA - Educational and Research Series

24 October 2026 | Edition 0.1 | Teaching manuscript

Contents

1 Goals, readiness, and learning route **2**

2 What is SPI? **2**
2.1 CPOL and CPHA: four operating modes 3

3 Mapping the protocol onto digital logic **4**
3.1 The byte-level hardware contract 4

4 Verilog: the operational reference **4**

5 SystemVerilog: expressing the same contract **5**

6 VHDL: equivalent behavior through explicit types **6**

7 Internal verification: the testbench **7**
7.1 SystemVerilog stimulus and assertions 8
7.2 VHDL stimulus and assertions 8

8 External orchestration: scripts and pass criteria **8**

9 Synthesis and real-board acceptance **9**
9.1 Reproducible research record 9

10 Comparing architectures: research beyond the exercise **9**

11 References, code and demonstration video **10**

> **Reader's note.** This booklet is an extensible technical teaching manuscript, not a certificate for every RTL variant described below. The **real reference** is a Verilog SPI Mode 0 master, simulated in SPI-01 and used in the successfully operated SDHC hardware probe. The SystemVerilog and VHDL extracts are **design sketches, not complete synthesizable releases**. For a reproducible project, download the complete source code rather than trying to rebuild it from excerpts in the PDF.

Reference source code: https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/examples/peripherals/spi/01-mode0-byte

Independent SD hardware implementation: https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/boards/tang-nano-9k/experiments/sd-sector0

Frozen code revision: `5b4ba625`. The `main` links are convenient for browsing but may evolve.
DOI: pending author-supplied publication identifier.

1 Goals, readiness, and learning route

After completing this module, a reader should be able to distinguish **bus signaling** from a device-specific command protocol; transmit and receive one SPI Mode 0 byte using programmable logic; verify edge placement, byte framing, and timing; and distinguish three levels of evidence: RTL simulation, FPGA synthesis and timing closure, and observation on a physical board.

Gate	What it establishes	Evidence as of 24 Oct 2026
SPI model	Four bytes transferred; 32 SCK rising edges counted	SPI-01 Icarus / CI PASS
Hardware integration	The byte engine participates in a physical SDHC exchange	Original LAB-05.1: operator-observed BOARD PASS
SystemVerilog / VHDL	Functional equivalence and synthesisability	Planned; **not independently tested**
Other SPI modes / devices	CPOL, CPHA, CS and electrical compatibility	**Not claimed**

This series deliberately separates a **general interface lesson** (`examples/peripherals/spi`) from a **specific board experiment** (`boards/tang-nano-9k/experiments/sd-sector0`). Either may later develop into a separate research track.

2 What is SPI?

The **Serial Peripheral Interface** is a synchronous serial link. A controller (master) generates the clock and selects a peripheral (slave). The traditional interface is **full duplex**: while one bit is driven on MOSI, another may be captured on MISO during the same SCK cycle. Unlike I2C, the usual four-wire SPI interface does not define a universal device address space or a common command vocabulary. The meaning of the transferred bytes belongs to the peripheral protocol.

Net	Meaning	FPGA responsibility
`SCK`	Serial clock	Generate the selected clock mode and sample edges
`MOSI`	Master-out, slave-in	Drive the next data bit before the required edge
`MISO`	Master-in, slave-out	Capture and assemble incoming data bits
`CS_n` / `SS_n`	Active-low peripheral select	Keep selected for the entire **device transaction**, possibly many bytes

One controller can share SCK/MOSI/MISO among peripherals while using a distinct select line for each. Peripheral requirements may include minimum CS setup and hold intervals. **The completion of a byte does not necessarily terminate a command:** the SD card remains selected during command, response, token, payload and CRC phases.



Figure 1: SPI design layers and signal ownership

2.1 CPOL and CPHA: four operating modes

CPOL selects the inactive level of SCK. **CPHA** selects whether input sampling occurs on the first or second clock edge of a bit period. Controller and peripheral must agree.

Mode	CPOL	CPHA	Sampling edge	Update / shift edge
0	0	0	Leading, rising	Trailing, falling
1	0	1	Second, falling	Leading, rising
2	1	0	Leading, falling	Trailing, rising
3	1	1	Second, rising	Leading, falling

The following waveform represents **our Mode 0** and `0xA5 = 1010_0101` sent MSB first. In this teaching test, `MISO = MOSI` forms a loopback connection; a real peripheral drives its own data.

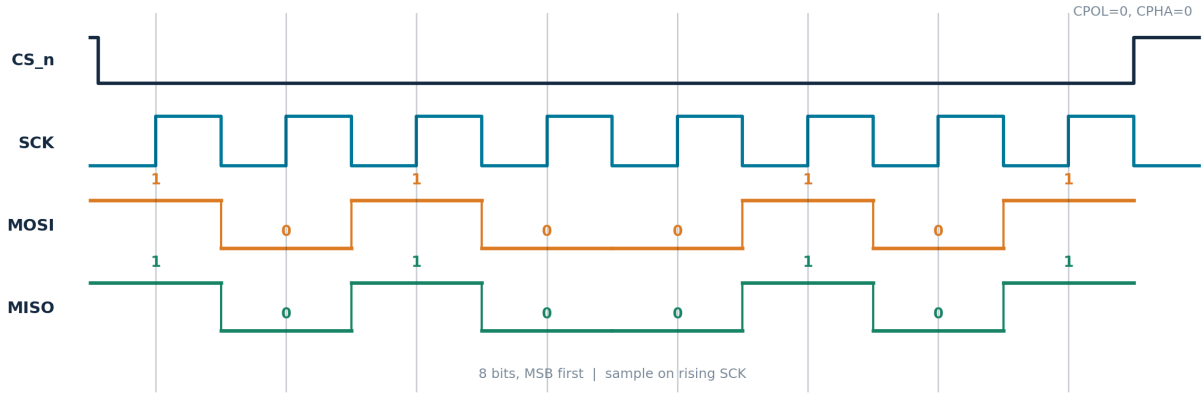


Figure 2: SPI Mode 0 timing, 0xA5, MSB first

The first MOSI bit **must already be valid before the first rising SCK edge**. Later bits change

on falling edges. Half-cycle misalignment is a common cause of shifted bytes or intermittent failures. Real-device setup and hold times must still be checked against the device datasheet.

3 Mapping the protocol onto digital logic

A minimal SPI byte master contains a clock divider, transmit holding register, receive shift/storage register, a bit counter and transaction-control logic. In this particular design `CS_n` belongs to the **higher-level device controller**, not to the byte engine. That allows a six-byte SD command and its reply to share one uninterrupted chip-select interval.

For the Tang Nano 9K implementation, the FPGA reference clock is 27 MHz and the Verilog parameter is `HALF_PERIOD = 64`:

$$f_{\mathrm{SCK}} = \frac{f_{\mathrm{clk}}}{2\,\mathtt{HALF_PERIOD}} = \frac{27\,000\,000}{128} = 210\,937.5\ \mathrm{Hz}.$$

Eight ideal SPI clock periods take approximately $8/f_{\mathrm{SCK}} = 37.93\ \mu\mathrm{s}$. A complete byte-engine request takes additional controller cycles for `start`, `busy` and `done` handshaking.

Clock architecture matters. Do not casually use a toggling output register as a second internal `always @(posedge sck)` domain. In SD-01 all storage and state machines use the 27 MHz system clock, while SCK is a **registered output waveform**. This avoids an extra fabric clock domain; it does **not** eliminate external MISO timing requirements.

3.1 The byte-level hardware contract

Signal	Direction	Invariant or expectation
`start`	input	One system-clock request, accepted while `busy=0`
`tx_data[7:0]`	input	Latched when the request is accepted
`miso`	input	Sampled on Mode 0 rising SCK edges
`sck`, `mosi`	output	Mode 0; MSB first; correct idle state
`busy`	output	Stays high while the byte is transferred
`done`	output	One-clock completion pulse
`rx_data[7:0]`	output	Complete received byte after `done`

The real module rejects invalid `HALF_PERIOD` values using a simulation-time guard. Such guards and counter widths should be included in any new implementation.

4 Verilog: the operational reference

The complete source is `lab05_spi_byte.v`, independently verified by its loopback testbench and used by the original physically tested SD probe. The snippets below are **illustrative excerpts**, not a substitute for that file.

```
module lab05_spi_byte #(
    parameter integer HALF_PERIOD = 64
```

```
) (
    input  wire clk, reset_n, start, miso,
    input  wire [7:0] tx_data,
    output reg sck, mosi, busy, done,
    output reg [7:0] rx_data
);
    reg [15:0] divider;
    reg high_phase;
    reg [2:0] bit_index;
    reg [7:0] tx_hold, rx_hold;
    // The complete sequential logic is in the linked source.
endmodule
```

Relevant operations inside its single sequential control block:

```
// Prepare the first bit BEFORE the first SCK rising edge.
if (!busy && start) begin
    busy      <= 1'b1;
    divider   <= HALF_PERIOD - 1;
    bit_index <= 3'd7;
    tx_hold   <= tx_data;
    rx_hold   <= 8'd0;
    mosi      <= tx_data[7];
end
// At an external SCK rising edge, collect the current MISO bit.
if (busy && divider == 0 && !high_phase) begin
    sck <= 1'b1;
    rx_hold[bit_index] <= miso;
end
```

These are **conceptual slices of mutually exclusive branches**. Do not paste them into separate sequential blocks or treat the truncated module as a ready FPGA design. The actual implementation contains counter updates, falling-edge output changes, reset and completion handling in a single coherent state machine.

5 SystemVerilog: expressing the same contract

SystemVerilog offers typed state enums, `logic`, `always_ff` and strong assertion capabilities. The underlying electrical protocol is unchanged; the benefit is clearer intent and a richer verification language.

```
// Architecture sketch only: not a complete SPI master.
typedef enum logic [1:0] {
  IDLE, SETUP, SAMPLE, SHIFT
} spi_phase_t;
spi_phase_t state;
logic [7:0] tx_q, rx_q;
```

```
logic [2:0] bit_index;
logic [15:0] div_count;

always_ff @(posedge clk or negedge reset_n) begin
  if (!reset_n) begin
    state <= IDLE;
    sck <= 1'b0;
  end else begin
    unique case (state)
      IDLE:   if (start) state <= SETUP;
      SETUP:  if (div_count == 0) state <= SAMPLE;
      SAMPLE: state <= SHIFT;
      SHIFT:  if (bit_index == 0) state <= IDLE;
              else state <= SETUP;
    endcase
  end
end
```

To become a complete synthesizable component, this needs divider logic, bit transfer, valid receive data, reset handling and a well-defined done pulse. **Equivalence with the Verilog reference must be demonstrated**, not assumed because the keywords look similar.

For simulation, SystemVerilog assertions can express interface invariants:

```
// Illustrative SVA in the testbench, not synthesizable FPGA logic.
property sck_idle_low;
  @(posedge clk) disable iff (!reset_n)
    !busy |-> !sck;
endproperty
assert property (sck_idle_low);
```

Some tools support different subsets of concurrent SVA; choose a simulator that supports the checks you actually use. A passing compile with assertions silently disabled is not a passing assertion test.

6 VHDL: equivalent behavior through explicit types

VHDL uses `std_logic`, `numeric_std` and constrained data types. The goal is not a line-for-line port of the Verilog syntax, but **identical pin-level timing and completion behavior** for the same requests.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- Fragment for the architecture declarative region, not a full entity.
type spi_phase_t is (idle, setup, sample, shift);
signal state     : spi_phase_t := idle;
```

```
signal bit_index : integer range 0 to 7 := 7;
signal tx_hold   : std_logic_vector(7 downto 0);
signal rx_hold   : std_logic_vector(7 downto 0);

process(clk, reset_n)
begin
  if reset_n = '0' then
    state <= idle;
    sck   <= '0';
  elsif rising_edge(clk) then
    case state is
      when idle   => if start = '1' then state <= setup; end if;
      when setup  => if divider = 0 then state <= sample; end if;
      when sample => state <= shift;
      when shift  => if bit_index = 0 then state <= idle;
                     else state <= setup; end if;
    end case;
  end if;
end process;
```

The sample intentionally omits port declarations, divider, shifts and response logic. It **cannot compile as a standalone VHDL design**. A full implementation must also define exact bit ordering (`7 downto 0`), reset semantics and timer-width constraints.

7 Internal verification: the testbench

A **testbench runs in the simulator**, not in the physical FPGA. It provides a clock, reset and stimulus, models the selected peripheral, and compares observed behavior with an expected reference. Board-level testing is a separate gate.

Our published `tb_lab05_spi_byte.v` transmits four bytes and checks all **32 rising edges**.

```
// Excerpt from the verification idea: loopback is only for this test.
wire miso = mosi;

// After each transaction, the testbench checks:
if (!done)        $fatal(1, "SPI timeout");
if (rx !== value) $fatal(1, "SPI data mismatch");
if (edges_after - edges_before != 8)
    $fatal(1, "Unexpected SCK edge count");
if (sck !== 1'b0) $fatal(1, "SCK must idle low");
```

Additional cases for a later revision: `start` while `busy` is asserted; reset mid-word; unexpected deselection; MISO stuck at 0 or 1; divisor boundary values; all CPOL/CPHA modes; multiple slaves. These are research tasks, **not claims that SPI-01 already passed them**.

7.1 SystemVerilog stimulus and assertions

```
// Outline only; send_byte must enforce a timeout and check its result.
initial begin
  reset_n = 0;
  repeat (4) @(negedge clk);
  reset_n = 1;
  send_byte(8'h00);
  send_byte(8'hFF);
  send_byte(8'hA5);
  send_byte(8'h3C);
  $display("SPI_SV_TB_PASS");
  $finish;
end
```

A complete SV testbench would supply `send_byte`, a clock generator, a scoreboard and assertions for the SCK pulse count and protocol invariants.

7.2 VHDL stimulus and assertions

```
-- Inside the VHDL testbench, after a bounded transaction:
assert rx_data = x"A5"
  report "SPI RX mismatch" severity failure;
assert sck = '0'
  report "SPI SCK not idle low" severity failure;
report "SPI_VHDL_TB_PASS" severity note;
```

An independent VHDL TB still requires a clock/reset driver, a peripheral behavioral model, an edge counter and a **finite watchdog**.

8 External orchestration: scripts and pass criteria

The external PowerShell script does **not** inspect analog voltages or electrical bits directly. It launches the tools, checks return codes and required PASS markers and can archive logs. This is a separate observation layer over the simulator and testbench.

For the published SPI-01 on Windows PowerShell 7:

```
git clone https://github.com/AIDevelopersMonster/Sipeed-FPGA.git
cd Sipeed-FPGA
& .\examples\peripherals\spi\01-mode0-byte\test.ps1
```

Expected output: `LAB05_SPI_BYTE_TB_PASS: 4 bytes, 32 clock pulses.`

Equivalent explicit Icarus commands:

```
$dir = '.\examples\peripherals\spi\01-mode0-byte'
iverilog -g2012 -s tb_lab05_spi_byte -o spi01.vvp `
    "$dir\rtl\spi_byte.v" "$dir\tests\tb_spi_byte.v"
if ($LASTEXITCODE -ne 0) { throw 'Compilation failed' }
```

```
vvp spi01.vvp
if ($LASTEXITCODE -ne 0) { throw 'Simulation failed' }
```

Templates for **future independently built implementations**:

```
# SystemVerilog, only after implementing complete RTL and TB:
iverilog -g2012 -s tb_spi_sv -o spi_sv.vvp spi_sv.sv tb_spi_sv.sv
vvp spi_sv.vvp

# VHDL using GHDL; again these source names are future targets:
ghdl -a --std=08 spi_master.vhd tb_spi_master.vhd
ghdl -e --std=08 tb_spi_master
ghdl -r --std=08 tb_spi_master --assert-level=error
```

These are workflow examples, not claims that `spi_master.vhd` or the other named SV/VHDL artifacts currently exist in `main`.

9 Synthesis and real-board acceptance

RTL PASS is not BOARD PASS. A successful RTL simulation does not verify physical pins, I/O voltage, signal integrity, MISO delays, PLL settings or post-route timing. A board-ready experiment needs a synthesizable top, pin and timing constraints, PnR and STA reports, and recorded observations on the actual hardware.

The original Tang Nano 9K SD-01 used the byte-level SPI as the physical layer underneath an SDHC protocol state machine. That is meaningful evidence for this **specific SPI Mode 0 integration**, not for every SPI mode or external device. Also distinguish the original observed hardware result from a new independently rebuilt snapshot; each bitstream needs its own provenance.

9.1 Reproducible research record

Field	What to preserve
Test article	FPGA board, PCB revision, attached peripheral
Source	Exact Git commit, licenses, input SHA-256
Toolchain	Icarus, GHDL, Gowin versions and host OS
Constraints	Clock rate, I/O standards, pin map, STA
Simulation	Scenarios, PASS/FAIL, saved VCD/waveforms
Hardware	Power supply, cables, terminal output and setup
Negative outcome	Smallest reproducible failing case and logs

10 Comparing architectures: research beyond the exercise

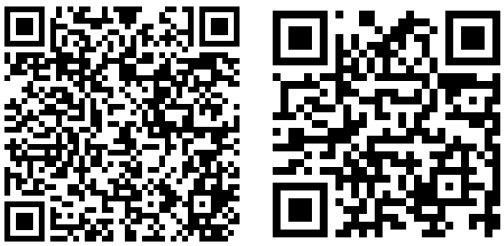
Future candidates include (1) the byte-level FSM, (2) a shift-register SPI master with FIFO, (3) vendor SPI IP, (4) CPU-controlled MMIO SPI, and (5) a streaming hardware-only SPI engine. Any comparison must freeze **the same transaction, clock, target FPGA, constraints and test set**.

Useful metrics: LUTs, FFs, BSRAM, Fmax, latency, power if measured, and verification complexity.

It would be unsound to conclude that a language is faster merely because its syntax differs. Synthesis settings, architecture, constraints and experiment protocol must be controlled before any measured ranking is published.

11 References, code and demonstration video

- **SPI-01 source:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/examples/peripherals/spi/01-mode0-byte
- **SD-01: real FPGA SPI, without a CPU:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/tree/main/boards/tang-nano-9k/experiments/sd-sector0
- **CI evidence for published source revision:** https://github.com/AIDevelopersMonster/Sipeed-FPGA/actions/runs/40052632010
- **Author's hardware demonstration video:** https://youtu.be/ybEUMzt7Gqg
- **Microchip SPI mode/timing background:** https://onlinedocs.microchip.com/oxy/GUID-6F1B86DC-C230-43D1-A405-0CC2E7EDF7EE-en-US-2/GUID-EAC2D943-F395-4D47-98C8-4EEBC798BFA2.html
- **SD Association simplified specifications:** https://www.sdcard.org/downloads/pls/



Edition 0.1 boundary: A coherent technical teaching basis, not a complete multi-language IP release. Future upgrades should supply compilable SV/VHDL designs, their testbenches, full waveform evidence for all four modes, and an isolated SPI-only board acceptance record.